4️⃣ Structured Query Language (SQL)

🧐 **Content**

1. Overview of SQL, Data Definition Commands, Integrity constraints: key constraints, Domain Constraints, Referential integrity , check constraints, Data Manipulation commands, Data Control commands, Set and string operations, aggregate function-group by, having, Views in SQL, joins, Nested and complex queries, Triggers

---

4.1 Overview of SQL

- **SQL (Structured Query Language)**: A standard language used to communicate with relational databases.
- Used for **creating**, **modifying**, **managing**, and **querying** data.

Data Definition Commands (DDL)

- Used to define the **structure** of the database.
- Common DDL commands:
  - `CREATE`: Create database objects (tables, views, etc.)
  - `ALTER`: Modify structure of database objects
  - `DROP`: Delete database objects
  - `TRUNCATE`: Remove all records from a table (faster than DELETE)

Integrity Constraints

Used to maintain **accuracy and consistency** of data in the database.

1. **Key Constraints**
   - **Primary Key**: Uniquely identifies each record; cannot be NULL.
   - **Unique Key**: Ensures all values in a column are unique.
   - **Foreign Key**: Links one table to another; enforces **referential integrity**.
2. **Domain Constraints**
   - Defines **valid values** for a column (e.g., data type, range).
3. **Referential Integrity**
   - Ensures that a **foreign key** value always refers to an existing, valid row in another table.
4. **Check Constraints**

- Used to enforce **specific conditions** (e.g., `CHECK (age >= 18)`).

Data Manipulation Commands (DML)

Used to manage **data within tables**:

- `SELECT`: Retrieve data
- `INSERT`: Add new data
- `UPDATE`: Modify existing data
- `DELETE`: Remove data

Data Control Commands (DCL)

Used for **permissions and access control**:

- `GRANT`: Give user access privileges
- `REVOKE`: Take back privileges

Set and String Operations

- **Set operations**:
  - `UNION`: Combine results of two queries (removes duplicates)
  - `UNION ALL`: Same as UNION, but includes duplicates
  - `INTERSECT`: Returns common records
  - `MINUS/EXCEPT`: Returns records from the first query not in the second
- **String operations**:
  - `CONCAT()`: Combine strings
  - `LIKE`, `SUBSTRING`, `LENGTH`, `UPPER`, `LOWER`

Aggregate Functions

Used to perform calculations on data:

- `COUNT()`, `SUM()`, `AVG()`, `MIN()`, `MAX()`

GROUP BY

- Groups rows sharing a property so aggregate functions can be applied.

HAVING

- Used to filter **groups** after grouping, often with aggregate functions.

Views in SQL

- **Virtual table** based on a query.
- Created using `CREATE VIEW`.
- Used for **abstraction, security, and simplification**.

Joins in SQL

Combines rows from two or more tables:

- **INNER JOIN**: Returns matching rows
- **LEFT JOIN**: All from left + matched from right
- **RIGHT JOIN**: All from right + matched from left
- **FULL OUTER JOIN**: All records with or without matches

Nested and Complex Queries

- **Nested/Subqueries**: Query within another query
  - Can be in `SELECT`, `FROM`, `WHERE`
- **Correlated subquery**: Depends on the outer query

Triggers

- A **procedure** that automatically runs when certain events occur in the database.
- Types:
  - **BEFORE INSERT/UPDATE/DELETE**
  - **AFTER INSERT/UPDATE/DELETE**
- Used for **enforcing rules, auditing, logging**.

Basic MySQL Queries

1. **SELECT Statement** – Used to retrieve data from a table.

```
SELECT column1, column2 FROM table_name;
```

2. **INSERT Statement** – Used to insert data into a table.

```
INSERT INTO table_name (column1, column2) VALUES ('value1', 'value2');
```

3. **UPDATE Statement** – Used to modify existing records.

```
UPDATE table_name SET column1 = 'new_value' WHERE condition;
```

4. **DELETE Statement** – Used to delete records from a table.

```
DELETE FROM table_name WHERE condition
```

2. Filtering Data

- **WHERE Clause** – Filters records based on conditions.

```
SELECT * FROM employees WHERE salary > 50000;
```

- **ORDER BY Clause** – Sorts the results.

```
SELECT * FROM students ORDER BY marks DESC;
```

- **LIMIT Clause** – Limits the number of records.

```
SELECT * FROM products LIMIT 5
```

3. Aggregate Functions

- **COUNT()** – Counts the number of rows.

```
SELECT COUNT(*) FROM orders;
```

- **SUM()** – Calculates the sum of a column.

```
SELECT SUM(price) FROM sales;
```

- **AVG()** – Calculates the average value.

```
SELECT AVG(marks) FROM students
```

- **GROUP BY Clause** – Groups records based on a column.

```
SELECT department, COUNT(*) FROM employees GROUP BY department;
```

4. GROUP BY and HAVING

- **GROUP BY** – Group rows by column

```
SELECT department, COUNT(*) FROM employees GROUP BY department;
```

- **HAVING** – Filter grouped results

```
SELECT department, AVG(salary)
FROM employees
GROUP BY department
HAVING AVG(salary) > 60000;
```

---

5. Joins in MySQL

- **INNER JOIN** – Returns records with matching values.

```
SELECT employees.name, departments.department_name
FROM employees
INNER JOIN departments ON employees.dept_id = departments.dept_id
```

- **LEFT JOIN** – Returns all records from the left table and matching records from the right.

```
SELECT students.name, courses.course_name
FROM students
LEFT JOIN courses ON students.course_id = courses.course_id;
```

- **RIGHT JOIN** – Returns all records from the right table and matching records from the left.

```
SELECT employees.name, projects.project_name
FROM employees
RIGHT JOIN projects ON employees.emp_id = projects.emp_id
```

6. Subqueries (A query inside another query)

- **In WHERE Clause**

```
SELECT name
FROM employees
WHERE salary > (SELECT AVG(salary) FROM employees);
```

- **In FROM Clause**

```
SELECT dept, avg_sal
FROM (
  SELECT department AS dept, AVG(salary) AS avg_sal
  FROM employees
  GROUP BY department
) AS dept_avg
WHERE avg_sal > 60000;
```

- **In SELECT Clause**

```
SELECT name,
  (SELECT COUNT(*) FROM tasks WHERE tasks.emp_id = employees.id) AS task_count
FROM employees;
```

7. Correlated Subqueries

- Subquery depends on the outer query

```
SELECT name
FROM employees e
WHERE salary > (
  SELECT AVG(salary)
  FROM employees
  WHERE department = e.department
);
```

8. Indexing

- Used to speed up searches.

```
CREATE INDEX idx_name ON employees(name);
```

8. Transactions

- Ensures data consistency.

```
START TRANSACTION;
UPDATE accounts SET balance = balance - 500 WHERE id = 1;
UPDATE accounts SET balance = balance + 500 WHERE id = 2;
COMMIT
```

9. Views

- A virtual table based on a query.

```
CREATE VIEW high_salary AS
SELECT name, salary FROM employees WHERE salary > 60000;
```

10. Stored Procedures

- Predefined SQL queries stored for execution.

```
CREATE PROCEDURE GetEmployees()
BEGIN
   SELECT * FROM employees;
END;
```

11. EXISTS and IN Operators

- **IN** – Check if value is in a list/subquery

```
SELECT name
FROM employees
WHERE dept_id IN (SELECT id FROM departments WHERE location = 'NY');
```

- **EXISTS** – Checks for existence of rows

```
SELECT name
FROM employees e
WHERE EXISTS (
  SELECT 1
  FROM tasks t
  WHERE t.emp_id = e.id
);
```